

Plano de estudos — Go para devs sênior

JS/TS + Python → GoLang · 5 fases · ~6 meses intensivos

Este plano assume experiência sólida em JavaScript/TypeScript e Python. O foco é queimar reflexos de outras linguagens e internalizar o jeito Go de resolver problemas: erros como valores, composição via interfaces, concorrência com goroutines e channels, e binários estáticos de alta performance.

Fase	Tema	Duração	Projeto prático
Fase 1	Fundamentos	Semanas 1–3	CLI de manipulação de arquivos CSV: lê, filtra, agrupa e exporta. Exer...
Fase 2	Concorrência	Semanas 4–7	Web crawler concorrente: busca URLs em paralelo com limite de workers
Fase 3	Backend real	Semanas 8–13	API REST completa: autenticação JWT, CRUD com PostgreSQL via SQ
Fase 4	Programação de sistemas	Semanas 14–20	Proxy HTTP reverso com balanceamento de carga, circuit breaker, métri
Fase 5	Expert	Semanas 21–26	Biblioteca Go open-source ou ferramenta de plataforma interna: API púb

Go vai te surpreender pela simplicidade deliberada. Sem herança, sem exceções, sem generics complexos. O objetivo é queimar os reflexos de JS/Python e começar a pensar o 'Go way': erros como valores, composição via interfaces, zero overhead.

Conteúdo por módulo

Sintaxe e tipos

- Variáveis, zero values e inferência de tipo (:=)
- Tipos básicos vs JS: int64, float64, rune, byte
- Arrays (valor!) vs slices (referência) vs maps
- Structs: composição em vez de herança
- Pointers — o que JS/Python escondem de você
- Funções como first-class values, closures

Error handling

- Erros como valores — por que não exceptions
- Idioma 'if err != nil' e o custo cognitivo
- Criando erros com errors.New e fmt.Errorf
- Wrapping e unwrapping com %w
- errors.Is e errors.As
- Erros customizados via interface

Interfaces e OOP Go-style

- Interfaces implícitas — duck typing estático
- Composição de interfaces
- Interface vazia 'any' e quando evitar
- Stringer, Reader, Writer — interfaces canônicas
- Embedding de structs
- Métodos em tipos não-struct

Toolchain e módulos

- go mod init, go.mod, go.sum
- go build, go run, go test
- GOPATH vs módulos (o que mudou em 1.11+)
- Formatação com gofmt / goimports
- Linting: golangci-lint
- Workspace com múltiplos módulos

Projeto da fase

CLI de manipulação de arquivos CSV: lê, filtra, agrupa e exporta. Exercita structs, slices, maps, error handling e I/O.

Dica pro

Dica de quem vem do TS: não existe undefined. Todo tipo tem um zero value (0, "", false, nil). Isso muda como você pensa em inicialização.

Recursos

go.dev/tour/ — Tour of Go · gobyexample.com — Go by Example · go.dev/doc/effective_go — Effective Go

Concorrência é onde Go brilha e se diferencia. Goroutines são baratas (2KB de stack inicial), channels são composáveis e o scheduler cooperativo evita o callback hell. Mas concorrência errada gera race conditions invisíveis — aqui você aprende a fazer certo.

Conteúdo por módulo

Goroutines e scheduler

- Goroutines vs threads do SO — diferença de custo
- O scheduler M:N do Go runtime
- Keyword 'go' — lançando goroutines
- GOMAXPROCS e paralelismo real
- Goroutine leaks — como detectar e evitar
- `runtime.Gosched()` e controle manual (raro)

Channels

- Channels unbuffered vs buffered
- `select` statement e multiplexação
- Fechar channels — quem fecha e por quê
- Pipeline pattern: produtor → transformador → consumidor
- Fan-out e fan-in patterns
- Done channels para cancelamento

sync e primitivos

- `sync.WaitGroup` — esperando goroutines
- `sync.Mutex` e `sync.RWMutex`
- `sync.Once` — inicialização única (singleton Go-style)
- `sync.Map` — quando usar vs `map` + `mutex`
- `atomic` package para operações atômicas
- Race detector: `go test -race`

Context

- `context.Context` — propagação de cancelamento
- `WithCancel`, `WithTimeout`, `WithDeadline`
- Passando context como primeiro argumento (idioma)
- Valores em context — quando usar e quando não
- Context em HTTP handlers e clients
- Padrão de shutdown graceful com context

Projeto da fase

Web crawler concorrente: busca URLs em paralelo com limite de workers (semáforo via channel), timeout por request e cancelamento via context.

Dica pro

Regra de ouro: 'Do not communicate by sharing memory; instead, share memory by communicating.' Channels primeiro, mutex quando channels forem awkward.

Recursos

go.dev/blog/concurrency-patterns — Concurrency Patterns · go.dev/blog/pipelines — Pipelines blog post

Aqui você constrói APIs, conecta bancos de dados e lida com os padrões de projeto que aparecem em todo codebase Go profissional. A stdlib é intencionalmente pequena — você vai aprender quando ela basta e quando puxar libs externas.

Conteúdo por módulo

HTTP e APIs REST

- net/http stdlib puro — mais capaz do que parece
- Handler, HandlerFunc, ServeMux (Go 1.22+)
- Middleware pattern com closures
- Chi ou Gin para routing mais complexo
- JSON encoding/decoding e struct tags
- Validação, binding e error responses padronizados

Banco de dados

- database/sql — interface padrão
- sqlx para scan mais ergonômico
- SQLC — gerar código Go tipado a partir de SQL
- GORM (e por que usar com moderação)
- Migrations com golang-migrate
- Connection pooling e configuração

Testes

- testing package — sem framework externo
- Table-driven tests — o idioma Go
- Mocks com interfaces (sem magic reflection)
- Testify para assertions e mocks
- Benchmarks com testing.B
- Fuzz testing (Go 1.18+)

Arquitetura Go

- Estrutura de projeto: flat vs hexagonal vs DDD
- Dependency injection manual (sem framework)
- Repository pattern com interfaces
- Errors: sentinel, typed, wrapped
- Configuração: flags, env vars, arquivos
- Logging estruturado com slog (Go 1.21+)

Projeto da fase

API REST completa: autenticação JWT, CRUD com PostgreSQL via SQLC, middleware de logging e rate limiting, testes com cobertura >80%.

Dica pro

Comparação com Node.js: o stdlib HTTP do Go já é production-ready — não precisa de Express para APIs simples. Adicione router externo só quando o ServeMux padrão for insuficiente.

Recursos

lets-go.alexedwards.net — Let's Go (livro) · github.com/sqlc-dev/sqlc — SQLC

Fase 4 Programação de sistemas

Semanas 14–20 · Go no nível do sistema

Go foi criado para sistemas — não só APIs. Aqui você entra em território que Python torna difícil e que JS simplesmente não faz: binários estáticos, interação com o SO, protocolos de rede de baixo nível e ferramentas de alta performance.

Conteúdo por módulo

Performance e profiling

- pprof: CPU, memória, goroutines, mutex
- Trace: visualizando o scheduler
- Escape analysis: stack vs heap
- Reduzindo alocações com sync.Pool
- Strings vs []byte — quando importa
- SIMD e unsafe pointer (uso avançado)

Rede e protocolos

- TCP/UDP raw com net package
- gRPC e Protobuf — alternativa ao REST
- WebSockets com gorilla/websocket
- HTTP/2 e TLS configuração manual
- DNS lookup e resolução customizada
- QUIC e HTTP/3 (quic-go)

CLI e ferramentas

- Cobra para CLIs complexas (estilo kubectl)
- Bubbletea para TUIs interativas
- Cross-compilation: GOOS=linux GOARCH=arm64
- Embed FS — embutir assets no binário
- CGo — interop com C (e por que evitar)
- Build tags e feature flags em tempo de compilação

Cloud native

- Containers: Dockerfile multi-stage para binários tiny
- Kubernetes operators com controller-runtime
- OpenTelemetry: traces, metrics, logs
- Health checks, graceful shutdown
- Feature flags e config dinâmica
- Secrets management via Vault ou AWS SSM

Projeto da fase

Proxy HTTP reverso com balanceamento de carga, circuit breaker, métricas Prometheus e tracing distribuído via OpenTelemetry.

Dica pro

Binário estático é superpoder: `CGO_ENABLED=0 go build` gera um executável sem dependências. Um Dockerfile FROM scratch com só o binário é <10MB e sem superfície de ataque.

Recursos

go.dev/blog/pprof — pprof guide · grpc.io/docs/languages/go/quickstart — gRPC Go quickstart

Você agora conhece Go. Essa fase é sobre dominar as partes que a maioria dos devs ignora: o runtime em profundidade, generics (1.18+), contribuir com o ecossistema e escrever Go que resiste a anos de manutenção em time grande.

Conteúdo por módulo

Generics (Go 1.18+)

- Type parameters e constraints
- Interface como constraint e como tipo
- comparable, any, union constraints
- slices, maps, cmp packages da stdlib
- Quando generics ajudam vs quando adicionam complexidade
- Implementando estruturas de dados genéricas

Runtime internals

- GC: tri-color mark-and-sweep, GOGC, GOMEMLIMIT
- Goroutine scheduling: G, M, P model
- Stack growth e shrink dinâmico
- Memory model do Go — happens-before
- unsafe package — leitura obrigatória
- Reflection com reflect — custo e uso correto

Patterns avançados

- Functional options pattern (Rob Pike)
- Options + Builder híbrido para APIs ergonômicas
- Iterator pattern com generics (Go 1.23 range-over-func)
- Event sourcing e CQRS em Go
- Plugin system com interfaces e dynamic loading
- Code generation com go generate

Go em time grande

- Módulos internos e replace directives
- API design: backward compatibility, deprecation
- Linting customizado com go/analysis
- Análise estática e AST manipulation
- Documentação: godoc, exemplos executáveis
- Semantic versioning e major version suffix

Projeto da fase

Biblioteca Go open-source ou ferramenta de plataforma interna: API pública estável, testes com fuzz, benchmarks documentados, CI com golangci-lint e release automatizado via goreleaser.

Dica pro

O maior salto de sênior Go: aprender a não usar generics quando interfaces bastam, não usar goroutines quando código sequencial é mais claro, não usar libs externas quando a stdlib resolve. Simplicidade é expertise.

Recursos

100go.co — 100 Go Mistakes · go.dev/ref/mem — Go Memory Model · github.com/goreleaser/goreleaser — goreleaser

Leitura obrigatória para expert

- 'The Go Programming Language' — Donovan & Kernighan
- '100 Go Mistakes and How to Avoid Them' — Teiva Harsanyi
- Proposals do GitHub golang/go — entender as decisões de design

Cronograma resumido

Fase	Semanas	Foco principal	Entregável
Fase 1	1–3	Sintaxe, tipos, toolchain, error handling	CLI CSV
Fase 2	4–7	Goroutines, channels, sync, context	Web crawler concorrente
Fase 3	8–13	HTTP, banco de dados, testes, arquitetura	API REST completa
Fase 4	14–20	Performance, rede, CLI, cloud native	Proxy HTTP reverso
Fase 5	21–26	Generics, runtime, patterns avançados	Lib open-source

Princípio final

O maior salto de sênior Go é aprender a não usar generics quando interfaces bastam, não usar goroutines quando código sequencial é mais claro, e não usar libs externas quando a stdlib resolve. Simplicidade é expertise — e é por isso que Go foi criado.